Chapter 6. The Date and Time API

In this chapter

- 6.1 The Time Line
- 6.2 Local Dates
- 6.3 Date Adjusters
- 6.4 Local Time
- 6.5 Zoned Time
- 6.6 Formatting and Parsing
- 6.7 Interoperating with Legacy Code

Time flies like an arrow, and we can easily set a starting point and count forward and backward in seconds. So why is it so hard to deal with time? The problem is humans. All would be easy if we could just tell each other: "Meet me at 1523793600, and don't be late!" But we want time to relate to daylight and the seasons. That's where things get complicated. Java 1.0 had a `Date` class that was, in hindsight, naïve, and had most of its methods deprecated in Java 1.1 when a `Calendar` class was introduced. Its API wasn't stellar, its instances were mutable, and it didn't deal with issues such as leap seconds. The third time is a charm, and the `java.time` API introduced in Java 8 has remedied the flaws of the past and should serve us for quite some time. In this chapter, you will learn what makes time computations so vexing, and how the Date and Time API solves these issues.

6.1 The Time Line

Historically, the fundamental time unit—the second—was derived from Earth's rotation around its axis. There are 24 hours or 24 × 60 × 60 = 86400 seconds in a full revolution, so it seems just a question of astronomical measurements to precisely define a sec-

ond. Unfortunately, Earth wobbles slightly, and a more precise definition was needed. In 1967, a new precise definition of a second, matching the historical definition, was derived from an intrinsic property of atoms of caesium-133. Since then, a network of atomic clocks keeps the official time.

Ever so often, the official time keepers synchronize the absolute time with the rotation of Earth. At first, the official seconds were slightly adjusted, but starting in 1972, "leap seconds" were occasionally inserted. (In theory, a second might need to be removed once in a while, but that has not yet happened.) There is talk of changing the system again. Clearly, leap seconds are a pain, and many computer systems instead use "smoothing" where time is artificially slowed down or sped up just before the leap second, keeping 86,400 seconds per day. This works because the local time on a computer isn't all that precise, and computers are used to synchronizing themselves with an external time service.

The Java Date and Time API specification requires that Java uses a time scale that

- Has 86,400 seconds per day
- Exactly matches the official time at noon each day
- Closely matches it elsewhere, in a precisely defined way

That gives Java the flexibility to adjust to future changes in the official time.

In Java, an `Instant` represents a point on the time line. An origin, called the *epoch*, is arbitrarily set at midnight of January 1, 1970 at the prime meridian that passes through the Greenwich Royal Observatory in London. This is the same convention used in the UNIX/POSIX time. Starting from that origin, time is measured in 86,400 seconds per day, forward and backward, to nanosecond precision. The `Instant` values go back as far as a billion years (`Instant.MIN`). That's not quite enough to express the age of the universe (around 13.5 billion years) but it should be enough for all practical purposes. After all, a billion years ago, the earth was covered in ice and populated by microscopic ancestors of today's plants and animals. The largest value, `Instant.MAX` , is December 31 of the year 1,000,000,000.

The static method call `Instant.now()` gives the current instant. You can compare two instants with the `equals` and `compareTo` methods in the usual way, so you can use instants as timestamps.

To find out the difference between two instants, use the static method `Duration.between`. For example, here is how you can measure the running time of an algorithm:

Click here to view code image

```
Instant start = Instant.now();
runAlgorithm();
Instant end = Instant.now();
Duration timeElapsed = Duration.between(start, end);
long millis = timeElapsed.toMillis();
```

A `Duration` is the amount of time between two instants. You can get the length of a `Duration` in conventional units by calling `toNanos`, `toMillis`, `toSeconds`, `toMinutes`, `toHours`, or `toDays`.



Note

In Java 8, you had to call `getSeconds` instead of `toSeconds`.

If you need nanosecond precision, be aware of overflow. A `long` value can hold almost 300 years of nanoseconds. If your durations are shorter than that, simply convert them to nanoseconds. You can use longer durations—a `Duration` object stores the number of seconds in a `long`, and the number of nanoseconds in an additional `int`. The `Duration` API has a number of methods, shown at the end of this section, for carrying out arithmetic.

For example, if you want to check whether an algorithm is at least ten times faster than another, you can compute

Click here to view code image

```
Duration timeElapsed2 = Duration.between(start2, end2);
boolean overTenTimesFaster
  = timeElapsed.multipliedBy(10).minus(timeElapsed2).isNegative();
```

This is just to show the syntax. Since the algorithms aren't going to run for hundreds of years, you can simply use

Click here to view code image

```
boolean overTenTimesFaster = timeElapsed.toNanos() * 10 < timeElapsed2.toNanos();
```



Note

The `Instant` and `Duration` classes are immutable, and all methods, such as `multipliedBy` or `minus`, return a new instance.

In the example program in **Listing 6.1**, you can see how to use the `Instant` and `Duration` classes for timing two algorithms.

Listing 6.1 timeline/TimeLine.java

Click here to view code image

```
package timeline;

/**
 * @version 1.01 2021-09-06
 * @author Cay Horstmann
 */

import java.time.*;
import java.util.*;
import java.util.stream.*;

public class Timeline
{
   public static void main(String[] args)
   {
      Instant start = Instant.now();
      runAlgorithm();
      Instant end = Instant.now();
      Duration timeElapsed = Duration.between(start, end);
      long millis = timeElapsed.toMillis();
      System.out.printf("%d milliseconds\n", millis);

      Instant start2 = Instant.now();
      runAlgorithm2();
      Instant end2 = Instant.now();
      Duration timeElapsed2 = Duration.between(start2, end2);
      System.out.printf("%d milliseconds\n", timeElapsed2.toMillis());
      boolean overTenTimesFaster = timeElapsed.multipliedBy(10)
        .minus(timeElapsed2).isNegative();
      System.out.printf("The first algorithm is %smore than ten times faster",
        overTenTimesFaster ? "" : "not ");
```

```
32     }
33
34     public static void runAlgorithm()
35     {
36        int size = 10;
37        ArrayList<Integer> list = new Random().ints().map(i -> i % 100).limit(size)
38           .boxed().collect(Collectors.toCollection(ArrayList::new));
39        Collections.sort(list);
40        System.out.println(list);
41     }
42
43     public static void runAlgorithm2()
44     {
45        int size = 10;
46        List<Integer> list = new Random().ints().map(i -> i % 100).limit(size)
47           .boxed().collect(Collectors.toCollection(ArrayList::new));
48        while (!IntStream.range(1, list.size())
49              .allMatch(i -> list.get(i - 1).compareTo(list.get(i)) <= 0))
50           Collections.shuffle(list);
51        System.out.println(list);
52     }
53  }
```

java.time.Instant 8

- `static Instant now()`
  gets the current instant from the best available system clock.
- `Instant plus(TemporalAmount amountToAdd)`
- `Instant minus(TemporalAmount amountToSubtract)`

yield an instant that is the given amount away from this `Instant`. The classes `Duration` and `Period` (see **Section 6.2**, "**Local Dates**," on p. **366**) implement the `TemporalAmount` interface.

- `Instant ( plus | minus )( Nanos | Millis | Seconds ) (long number)`
  yields an `Instant` that is the given number of nanoseconds, milliseconds, or seconds away from this `Instant`.

`java.time.Duration` 8

- `static Duration of ( Nanos | Millis | Seconds | Minutes | Hours | Days ) (long number)`
  yields a duration of the given number of time units.
- `static Duration between(Temporal startInclusive, Temporal endExclusive)`
  yields a duration between the given points in time. The `Temporal` interface is implemented by the `Instant` class as well as `LocalDate` / `LocalDateTime` / `LocalTime` (see **Section 6.4**, "**Local Time**," on p. **373**) and `ZonedDateTime` (see **Section 6.5**, "**Zoned Time**," on p. **375**).
- `long toNanos()`
- `long toMillis()`
- `long toSeconds()` 9
- `long toMinutes()`
- `long toHours()`
- `long toDays()`
  get the number of the time units in the method name for this `Duration`.
- `int to ( Nanos | Millis | Seconds | Minutes | Hours ) Part()` 9
- `long toDaysPart()` 9
  The part of the given time unit in this `Duration`. For example, in a duration of 100 seconds, the minutes part is `1` and the seconds part is `40`.
- `Duration plus(TemporalAmount amountToAdd)`
- `Duration minus(TemporalAmount amountToSubtract)`

yield a duration that is the given amount away from this `Duration`. The classes `Duration` and `Period` (see **Section 6.2**, "**Local Dates**," on p. **366**) implement the `TemporalAmount` interface.

- `Duration multipliedBy(long multiplicand)`
- `Duration dividedBy(long divisor)`
- `Duration negated()`
  yield a duration that is obtained by multiplying or dividing this `Duration` by the given amount, or by –1.
- `boolean isZero()`
- `boolean isNegative()`
  return `true` if this `Duration` is zero or negative.
- `Duration ( plus | minus )( Nanos | Millis | Seconds | Minutes | Hours | Days ) (long number)`
  yields a `Duration` obtained by adding or subtracting the given number of time units.

6.2 Local Dates

Now let us turn from absolute time to human time. There are two kinds of human time in the Java API, *local date/time* and *zoned time*. Local date/time has a date and/or time of day, but no associated time zone information. An example of a local date is June 14, 1903 (the day on which Alonzo Church, inventor of the lambda calculus, was born). Since that date has neither a time of day nor time zone information, it does not correspond to a precise instant of time. In contrast, July 16, 1969, 09:32:00 EDT (the launch of Apollo 11) is a zoned date/time, representing a precise instant on the time line.

There are many calculations where time zones are not required, and in some cases they can even be a hindrance. Suppose you schedule a meeting every week at 10:00. If you add 7 days (that is, 7 × 24 × 60 × 60 seconds) to the last zoned time, and you happen to cross the daylight savings time boundary, the meeting will be an hour too early or too late!

For that reason, the API designers recommend that you do not use zoned time unless you really want to represent absolute time instances. Birthdays, holidays, schedule times, and so on are usually best represented as local dates or times.

A `LocalDate` is a date with a year, month, and day of the month. To construct one, you can use the `now` or `of` static methods:

Click here to view code image

```
LocalDate today = LocalDate.now(); // Today's date
LocalDate alonzosBirthday = LocalDate.of(1903, 6, 14);
alonzosBirthday = LocalDate.of(1903, Month.JUNE, 14);
  // Uses the Month enumeration
```

Unlike the irregular conventions in UNIX and `java.util.Date`, where months are zero-based and years are counted from 1900, here you supply the usual numbers for the month of year. Alternatively, you can use the `Month` enumeration.

The API notes at the end of this section show the most useful methods for working with `LocalDate` objects.

For example, *Programmer's Day* is the 256th day of the year. Here is how you can easily compute it:

Click here to view code image

```
LocalDate programmersDay = LocalDate.of(2014, 1, 1).plusDays(255);
  // September 13, but in a leap year it would be September 12
```

Recall that the difference between two time instants is a `Duration`. The equivalent for local dates is a `Period`, which expresses a number of elapsed years, months, or days. You can call `birthday.plus(Period.ofYears(1))` to get the birthday next year. Of course, you can also just call `birthday.plusYears(1)`. But `birthday.plus(Period.ofDays(365))` won't produce the correct result in a leap year.

The `until` method yields the difference between two local dates. For example,

Click here to view code image

```
independenceDay.until(christmas)
```

yields a period of 5 months and 21 days. That is actually not terribly useful because the number of days per month varies. To find the number of days, use

Click here to view code image

```
independenceDay.until(christmas, ChronoUnit.DAYS) // 174 days
```



Caution

Some methods in the `LocalDate` API could potentially create nonexistent dates. For example, adding one month to January 31 should not yield February 31. Instead of throwing an exception, these methods return the last valid day of the month. For example,

Click here to view code image

```
LocalDate.of(2016, 1, 31).plusMonths(1)
```

and

Click here to view code image

```
LocalDate.of(2016, 3, 31).minusMonths(1)
```

yield February 29, 2016.

The `getDayOfWeek` yields the weekday, as a value of the `DayOfWeek` enumeration. `DayOfWeek.MONDAY` has the numerical value `1`, and `DayOfWeek.SUNDAY` has the value `7`. For example,

Click here to view code image

```
LocalDate.of(1900, 1, 1).getDayOfWeek().getValue()
```

yields `1`. The `DayOfWeek` enumeration has convenience methods `plus` and `minus` to compute weekdays modulo `7`. For example, `DayOfWeek.SATURDAY.plus(3)` yields `DayOfWeek.TUESDAY`.



Note

The weekend days actually come at the end of the week. This is different from `java.util.Calendar` where Sunday has value `1` and Saturday value `7`.

Java 9 adds two useful methods `datesUntil` that yield streams of `LocalDate` objects.

Click here to view code image

```
LocalDate start = LocalDate.of(2000, 1, 1);
LocalDate endExclusive = LocalDate.now();
Stream<LocalDate> allDays = start.datesUntil(endExclusive);
Stream<LocalDate> firstDaysInMonth = start.datesUntil(endExclusive, Period.ofMonths(1));
```

In addition to `LocalDate`, there are also classes `MonthDay`, `YearMonth`, and `Year` to describe partial dates. For example, December 25 (with the year unspecified) can be represented as a `MonthDay`.

The example program in **Listing 6.2** shows how to work with the `LocalDate` class.

Listing 6.2 localdates/LocalDates.java

Click here to view code image

```
1  package localdates;
2
3  /**
4   * @version 1.01 2021-09-06
5   * @author Cay Horstmann
6   */
7  import java.time.*;
8  import java.time.temporal.*;
9  import java.util.stream.*;
```

```
10
11 public class LocalDates
12 {
13    public static void main(String[] args)
14    {
15       LocalDate today = LocalDate.now(); // Today's date
16       System.out.println("today: " + today);
17
18       LocalDate alonzosBirthday = LocalDate.of(1903, 6, 14);
19       alonzosBirthday = LocalDate.of(1903, Month.JUNE, 14);
20       // Uses the Month enumeration
21       System.out.println("alonzosBirthday: " + alonzosBirthday);
22
23       LocalDate programmersDay = LocalDate.of(2018, 1, 1).plusDays(255);
24       // September 13, but in a leap year it would be September 12
25       System.out.println("programmersDay: " + programmersDay);
26
27       LocalDate independenceDay = LocalDate.of(2018, Month.JULY, 4);
28       LocalDate christmas = LocalDate.of(2018, Month.DECEMBER, 25);
29
30       System.out.println("Until christmas: " + independenceDay.until(christmas));
31       System.out.println("Until christmas: "
32          + independenceDay.until(christmas, ChronoUnit.DAYS));
33
34       System.out.println(LocalDate.of(2016, 1, 31).plusMonths(1));
35       System.out.println(LocalDate.of(2016, 3, 31).minusMonths(1));
36
37       DayOfWeek startOfLastMillennium = LocalDate.of(1900, 1, 1).getDayOfWeek();
38       System.out.println("startOfLastMillennium: " + startOfLastMillennium);
39       System.out.println(startOfLastMillennium.getValue());
40       System.out.println(DayOfWeek.SATURDAY.plus(3));
41
42       LocalDate start = LocalDate.of(2000, 1, 1);
```

```
43        LocalDate endExclusive = LocalDate.now();
44        Stream<LocalDate> firstDaysInMonth = start.datesUntil(endExclusive, Period.ofMonths(1)
45        System.out.println("firstDaysInMonth: "
46           + firstDaysInMonth.toList());
47     }
48  }
```

java.time.LocalDate 8

- `static LocalDate now()`
  gets the current `LocalDate`.
- `static LocalDate of(int year, int month, int dayOfMonth)`
- `static LocalDate of(int year, Month month, int dayOfMonth)`
  yield a local date with the given year, month (as integer between 1 and 12 or a value of the `Month` enumeration), and day of the month (between 1 and 31).
- `LocalDate` `(` `plus` | `minus` `)(` `Days` | `Weeks` | `Months` | `Years` `) (long number)`
  yields a `LocalDate` obtained by adding or subtracting the given number of time units.
- `LocalDate plus(TemporalAmount amountToAdd)`
- `LocalDate minus(TemporalAmount amountToSubtract)`
  yields a `LocalDate` that is the given amount away from this `LocalDate`. The classes `Duration` and `Period` implement the `TemporalAmount` interface.
- `LocalDate withDayOfMonth(int dayOfMonth)`
- `LocalDate withDayOfYear(int dayOfYear)`
- `LocalDate withMonth(int month)`
- `LocalDate withYear(int year)`
  return a new `LocalDate` with the day of month, day of year, month, or year changed to the given value.
- `int getDayOfMonth()`

gets the day of the month (between 1 and 31).

- `int getDayOfYear()`
  gets the day of the year (between 1 and 366).
- `DayOfWeek getDayOfWeek()`
  gets the day of the week, returning a value of the `DayOfWeek` enumeration.
- `Month getMonth()`
- `int getMonthValue()`
  get the month as a value of the `Month` enumeration, or as a number between 1 and 12.
- `int getYear()`
  gets the year, between -999,999,999 and 999,999,999.
- `Period until(ChronoLocalDate endDateExclusive)`
  gets the period until the given end date. The `ChronoLocalDate` interface is implemented by `LocalDate` and date classes for non-Gregorian calendars.
- `boolean isBefore(ChronoLocalDate other)`
- `boolean isAfter(ChronoLocalDate other)`
  return `true` if this date is before or after the given date.
- `boolean isLeapYear()`
  returns `true` if the year is a leap year—that is, if it is divisible by 4 but not by 100, or divisible by 400. The algorithm is applied for all past years, even though that is historically inaccurate. (Leap years were invented in the year -46, and the rules involving divisibility by 100 and 400 were introduced in the Gregorian calendar reform of 1582. The reform took over 300 years to become universal.)
- `Stream<LocalDate> datesUntil(LocalDate endExclusive)` 9
- `Stream<LocalDate> datesUntil(LocalDate endExclusive, Period step)` 9
  yield a stream of dates from this `LocalDate` until the end, with step size 1 or the given period.

---

java.time.Period 8

- static Period of(int years, int months, int days)
- Period of (Days | Weeks | Months | Years) (int number)
  yield a Period with the given number of time units.
- int get (Days | Months | Years) ()
  gets the days, months, or years of this Period .
- Period (plus | minus)(Days | Months | Years) (long number)
  yields a Period obtained by adding or subtracting the given number of time units.
- Period plus(TemporalAmount amountToAdd)
- Period minus(TemporalAmount amountToSubtract)
  yield an instant that is the given amount away from this Instant . The classes Duration and Period implement the TemporalAmount interface.
- Period with (Days | Months | Years) (int number)
  returns a new Period with the days, months, or years changed to the given number.

6.3 Date Adjusters

For scheduling applications, you often need to compute dates such as "the first Tuesday of every month." The TemporalAdjusters class provides a number of static methods for common adjustments. You pass the result of an adjustment method to the with method. For example, the first Tuesday of a month can be computed like this:

Click here to view code image

```
LocalDate firstTuesday = LocalDate.of(year, month, 1).with(
  TemporalAdjusters.nextOrSame(DayOfWeek.TUESDAY));
```

As always, the `with` method returns a new `LocalDate` object without modifying the original. The API notes at the end of this section show the available adjusters.

You can also make your own adjuster by implementing the `TemporalAdjuster` interface. Here is an adjuster for computing the next weekday:

Click here to view code image

```
TemporalAdjuster NEXT_WORKDAY = w ->
  {
    var result = (LocalDate) w;
    do
    {
      result = result.plusDays(1);
    }
    while (result.getDayOfWeek().getValue() >= 6);
    return result;
  };
LocalDate backToWork = today.with(NEXT_WORKDAY);
```

Note that the parameter of the lambda expression has type `Temporal`, and it must be cast to `LocalDate`. You can avoid this cast with the `ofDateAdjuster` method that expects a lambda of type `UnaryOperator<LocalDate>`.

Click here to view code image

```
TemporalAdjuster NEXT_WORKDAY = TemporalAdjusters.ofDateAdjuster(w ->
  {
```

```
    LocalDate result = w; // No cast
    do
    {
      result = result.plusDays(1);
    }
    while (result.getDayOfWeek().getValue() >= 6);
    return result;
  });
```

java.time.LocalDate 9

- `LocalDate with(TemporalAdjuster adjuster)`
  returns the result of adjusting this date by the given adjuster.

java.time.temporal.TemporalAdjusters 9

- `static TemporalAdjuster next(DayOfWeek dayOfWeek)`
- `static TemporalAdjuster nextOrSame(DayOfWeek dayOfWeek)`
- `static TemporalAdjuster previous(DayOfWeek dayOfWeek)`
- `static TemporalAdjuster previousOrSame(DayOfWeek dayOfWeek)`
  return an adjuster that adjusts a date to the given day of the week.
- `static TemporalAdjuster dayOfWeekInMonth(int n, DayOfWeek dayOfWeek)`
- `static TemporalAdjuster lastInMonth(DayOfWeek dayOfWeek)`
  return an adjuster that adjusts a date to the `n` th or last given weekday on the month.
- `static TemporalAdjuster firstDayOfMonth()`
- `static TemporalAdjuster firstDayOfNextMonth()`

- `static TemporalAdjuster firstDayOfYear()`
- `static TemporalAdjuster firstDayOfNextYear()`
- `static TemporalAdjuster lastDayOfMonth()`
- `static TemporalAdjuster lastDayOfYear()`

  return an adjuster that adjusts a date to the given day of the month or year.

6.4 Local Time

A `LocalTime` represents a time of day, such as 15:30:00. You can create an instance with the `now` or `of` methods:

Click here to view code image

```
LocalTime rightNow = LocalTime.now();
LocalTime bedtime = LocalTime.of(22, 30); // or LocalTime.of(22, 30, 0)
```

The API notes show common operations with local times. The `plus` and `minus` operations wrap around a 24-hour day. For example,

Click here to view code image

```
LocalTime wakeup = bedtime.plusHours(8); // wakeup is 6:30:00
```



Note

`LocalTime` doesn't concern itself with AM/PM. That silliness is left to a formatter—see **Section 6.6**, "**Formatting and Parsing**," on p. **379**.

There is a `LocalDateTime` class representing a date and time. That class is suitable for storing points in time in a fixed time zone—for example, for a schedule of classes or events. However, if you need to make calculations that span the daylight savings time, or if you need to deal with users in different time zones, you should use the `ZonedDateTime` class that we discuss next.

`java.time.LocalTime` 8

- `static LocalTime now()`
  gets the current `LocalTime`.
- `static LocalTime of(int hour, int minute)`
- `static LocalTime of(int hour, int minute, int second)`
- `static LocalTime of(int hour, int minute, int second, int nanoOfSecond)`
  yield a local time with the given hour (between 0 and 23), minute, second (between 0 and 59), and nanosecond (between 0 and 999,999,999).
- `LocalTime ( plus | minus )( Hours | Minutes | Seconds | Nanos ) (long number)`
  yields a `LocalTime` obtained by adding or subtracting the given number of time units.
- `LocalTime plus(TemporalAmount amountToAdd)`
- `LocalTime minus(TemporalAmount amountToSubtract)`
  yield a `LocalTime` that is the given amount away from this `LocalTime`.
- `LocalTime with ( Hour | Minute | Second | Nano ) (int value)`
  returns a new `LocalTime` with the hour, minute, second, or nanosecond changed to the given value.
- `int getHour()`

gets the hour (between 0 and 23).

- `int getMinute()`
- `int getSecond()`

  get the minute or second (between 0 and 59).
- `int getNano()`

  gets the nanoseconds (between 0 and 999,999,999).
- `int toSecondOfDay()`
- `long toNanoOfDay()`

  yield the seconds or nanoseconds since midnight.
- `boolean isBefore(LocalTime other)`
- `boolean isAfter(LocalTime other)`

  return `true` if this date is before or after the given date.

6.5 Zoned Time

Time zones, perhaps because they are an entirely human creation, are even messier than the complications caused by the earth's irregular rotation. In a rational world, we'd all follow the clock in Greenwich, and some of us would eat our lunch at 02:00, others at 22:00. Our stomachs would figure it out. This is actually done in China, which spans four conventional time zones. Elsewhere, we have time zones with irregular and shifting boundaries and, to make matters worse, the daylight savings time.

As capricious as the time zones may appear to the enlightened, they are a fact of life. When you implement a calendar application, it needs to work for people who fly from one country to another. When you have a conference call at 10:00 in New York, but happen to be in Berlin, you expect to be alerted at the correct local time.

The Internet Assigned Numbers Authority (IANA) keeps a database of all known time zones around the world (**www.iana.org/time-zones**), which is updated several times per year. The bulk of the updates deals with the changing rules for daylight savings time. Java uses the IANA database.

Each time zone has an ID, such as `America/New_York` or `Europe/Berlin`. To find out all available time zones, call `ZoneId.getAvailableZoneIds`. At the time of this writing, there are almost 600 IDs.

Given a time zone ID, the static method `ZoneId.of(id)` yields a `ZoneId` object. You can use that object to turn a `LocalDateTime` object into a `ZonedDateTime` object by calling `local.atZone(zoneId)`, or you can construct a `ZonedDateTime` by calling the static method `ZonedDateTime.of(year, month, day, hour, minute, second, nano, zoneId)`. For example,

Click here to view code image

```
ZonedDateTime apollo11launch  = ZonedDateTime.of(1969, 7, 16, 9, 32, 0, 0,
  ZoneId.of("America/New_York"));
  // 1969-07-16T09:32-04:00[America/New_York]
```

This is a specific instant in time. Call `apollo11launch.toInstant` to get the `Instant`. Conversely, if you have an instant in time, call `instant.atZone(ZoneId.of("UTC"))` to get the `ZonedDateTime` at the Greenwich Royal Observatory, or use another `ZoneId` to get it elsewhere on the planet.



Note

UTC stands for “Coordinated Universal Time,” and the acronym is a compromise between the aforementioned English and the French “Temps Universel Coordiné,” having the distinction of being incorrect in either language. UTC is the time at the Greenwich Royal Observatory, without daylight savings time.

Many of the methods of `ZonedDateTime` are the same as those of `LocalDateTime` (see the API notes at the end of this section). Most are straightforward, but daylight savings time introduces some complications.

When daylight savings time starts, clocks advance by an hour. What happens when you construct a time that falls into the skipped hour? For example, in 2013, Central Europe switched to daylight savings time on March 31 at 2:00. If you try to construct nonexistent time March 31 2:30, you actually get 3:30.

Click here to view code image

```
ZonedDateTime skipped = ZonedDateTime.of(
  LocalDate.of(2013, 3, 31),
  LocalTime.of(2, 30),
  ZoneId.of("Europe/Berlin"));
  // Constructs March 31 3:30
```

Conversely, when daylight time ends, clocks are set back by an hour, and there are two instants with the same local time! When you construct a time within that span, you get the earlier of the two.

Click here to view code image

```
ZonedDateTime ambiguous = ZonedDateTime.of(
  LocalDate.of(2013, 10, 27), // End of daylight savings time
  LocalTime.of(2, 30),
  ZoneId.of("Europe/Berlin"));
  // 2013-10-27T02:30+02:00[Europe/Berlin]
ZonedDateTime anHourLater = ambiguous.plusHours(1);
  // 2013-10-27T02:30+01:00[Europe/Berlin]
```

An hour later, the time has the same hours and minutes, but the zone offset has changed.

You also need to pay attention when adjusting a date across daylight savings time boundaries. For example, if you set a meeting for next week, don't add a duration of seven days:

Click here to view code image

```
ZonedDateTime nextMeeting = meeting.plus(Duration.ofDays(7));
  // Caution! Won't work with daylight savings time
```

Instead, use the `Period` class.

Click here to view code image

```
ZonedDateTime nextMeeting = meeting.plus(Period.ofDays(7)); // OK
```



Caution

There is also an `OffsetDateTime` class that represents times with an offset from UTC, but without time zone rules. That class is intended for specialized applications that specifically require the absence of those rules, such as certain network protocols. For human time, use `ZonedDateTime`.

The example program in **Listing 6.3** demonstrates the `ZonedDateTime` class.

Listing 6.3 zonedtimes/ZonedTimes.java

Click here to view code image

```
1  package zonedtimes;
2
3  /**
4   * @version 1.0 2016-05-10
5   * @author Cay Horstmann
6   */
7
8  import java.time.*;
9
10 public class ZonedTimes
11 {
12    public static void main(String[] args)
13    {
14       ZonedDateTime apollo11launch = ZonedDateTime.of(1969, 7, 16, 9, 32, 0, 0,
15          ZoneId.of("America/New_York")); // 1969-07-16T09:32-04:00[America/New_York]
16       System.out.println("apollo11launch: " + apollo11launch);
17
18       Instant instant = apollo11launch.toInstant();
19       System.out.println("instant: " + instant);
20
21       ZonedDateTime zonedDateTime = instant.atZone(ZoneId.of("UTC"));
22       System.out.println("zonedDateTime: " + zonedDateTime);
23
24       ZonedDateTime skipped = ZonedDateTime.of(LocalDate.of(2013, 3, 31),
25          LocalTime.of(2, 30), ZoneId.of("Europe/Berlin")); // Constructs March 31 3:30
26       System.out.println("skipped: " + skipped);
27
```

```
28        ZonedDateTime ambiguous = ZonedDateTime.of(
29           LocalDate.of(2013, 10, 27), // End of daylight savings time
30           LocalTime.of(2, 30), ZoneId.of("Europe/Berlin"));
31           // 2013-10-27T02:30+02:00[Europe/Berlin]
32        ZonedDateTime anHourLater = ambiguous.plusHours(1);
33           // 2013-10-27T02:30+01:00[Europe/Berlin]
34        System.out.println("ambiguous: " + ambiguous);
35        System.out.println("anHourLater: " + anHourLater);
36
37        ZonedDateTime meeting = ZonedDateTime.of(LocalDate.of(2013, 10, 31),
38           LocalTime.of(14, 30), ZoneId.of("America/Los_Angeles"));
39        System.out.println("meeting: " + meeting);
40        ZonedDateTime nextMeeting = meeting.plus(Duration.ofDays(7));
41           // Caution! Won't work with daylight savings time
42        System.out.println("nextMeeting: " + nextMeeting);
43        nextMeeting = meeting.plus(Period.ofDays(7)); // OK
44        System.out.println("nextMeeting: " + nextMeeting);
45     }
46  }
```

java.time.ZonedDateTime 8

- `static ZonedDateTime now()`
  gets the current `ZonedDateTime`.
- `static ZonedDateTime of(int year, int month, int dayOfMonth, int hour, int minute, int second, int nanoOfSecond, ZoneId zone)`
- `static ZonedDateTime of(LocalDate date, LocalTime time, ZoneId zone)`
- `static ZonedDateTime of(LocalDateTime localDateTime, ZoneId zone)`
- `static ZonedDateTime ofInstant(Instant instant, ZoneId zone)`

yield a `ZonedDateTime` with the given parameters and time zone.

- `ZonedDateTime ( plus | minus )( Days | Weeks | Months | Years | Hours | Minutes | Seconds | Nanos ) (long number)`
  yields a `ZonedDateTime` obtained by adding or subtracting the given number of time units.
- `ZonedDateTime plus(TemporalAmount amountToAdd)`
- `ZonedDateTime minus(TemporalAmount amountToSubtract)`
  yield a `ZonedDateTime` that is the given amount away from this `ZonedDateTime`.
- `ZonedDateTime with ( DayOfMonth | DayOfYear | Month | Year | Hour | Minute | Second | Nano ) (int value)`
  returns a new `ZonedDateTime` with the given temporal unit replaced by the given value.
- `ZonedDateTime withZoneSameInstant(ZoneId zone)`
- `ZonedDateTime withZoneSameLocal(ZoneId zone)`
  return a new `ZonedDateTime` in the given time zone, either representing the same instant or the same local time.
- `int getDayOfMonth()`
  gets the day of the month (between 1 and 31).
- `int getDayOfYear()`
  gets the day of the year (between 1 and 366).
- `DayOfWeek getDayOfWeek()`
  gets the day of the week, returning a value of the `DayOfWeek` enumeration.
- `Month getMonth()`
- `int getMonthValue()`
  get the month as a value of the `Month` enumeration, or as a number between 1 and 12.
- `int getYear()`
  gets the year, between -999,999,999 and 999,999,999.
- `int getHour()`
  gets the hour (between 0 and 23).
- `int getMinute()`
- `int getSecond()`
  get the minute or second (between 0 and 59).
- `int getNano()`

gets the nanoseconds (between 0 and 999,999,999).

- `public ZoneOffset getOffset()`
  gets the offset from UTC. Offsets can vary from –12:00 to +14:00. Some time zones have fractional offsets. Offsets change with daylight savings time.
- `LocalDate toLocalDate()`
- `LocalTime toLocalTime()`
- `LocalDateTime toLocalDateTime()`
- `Instant toInstant()`
  yield the local date, time, or date/time, or the corresponding instant.
- `boolean isBefore(ChronoZonedDateTime other)`
- `boolean isAfter(ChronoZonedDateTime other)`
  return `true` if this zoned date/time is before or after the given zoned date/time.

6.6 Formatting and Parsing

The `DateTimeFormatter` class provides three kinds of formatters to print a date/time value:

- Predefined standard formatters (see **Table 6.1**)

Table 6.1 Predefined Formatters

Formatter	Description	Example
`BASIC_ISO_DATE`	Year, month, day, zone offset without separators	`19690716-0500`
`ISO_LOCAL_DATE`, `ISO_LOCAL_TIME`, `ISO_LOCAL_`	Separators `-`, `:`, `T`	`1969-07-16`, `09:32:00`, `1969-07-16T09:32:00`

Formatter	Description	Example
DATE_TIME		
ISO_OFFSET_DATE , ISO_OFFSET_TIME , ISO_OFFSET_DATE_TIME	Like ISO_LOCAL_*XXX* , but with zone offset	1969-07-16-05:00 , 09:32:00-05:00 , 1969-07-16T09:32:00-05:00
ISO_ZONED_DATE_TIME	With zone offset and zone ID	1969-07-16T09:32:00-05:00[America/New_York]
ISO_INSTANT	In UTC, denoted by the Z zone ID	1969-07-16T14:32:00Z
ISO_DATE , ISO_TIME , ISO_DATE_TIME	Like ISO_OFFSET_DATE , ISO_OFFSET_TIME , and ISO_ZONED_DATE_TIME , but the zone information is optional	1969-07-16-05:00 , 09:32:00-05:00 , 1969-07-16T09:32:00-05:00[America/New_York]
ISO_ORDINAL_DATE	The year and day of year, for LocalDate	1969-197
ISO_WEEK_DATE	The year, week, and day of week, for LocalDate	1969-W29-3
RFC_1123_DATE_TIME	The standard for email timestamps, codified in RFC 822 and updated to four digits for the year in RFC 1123	Wed, 16 Jul 1969 09:32:00 -0500

- Locale-specific formatters
- Formatters with custom patterns

To use one of the standard formatters, simply call its `format` method:

Click here to view code image

```
String formatted = DateTimeFormatter.ISO_OFFSET_DATE_TIME.format(apollo11launch);
  // 1969-07-16T09:32:00-04:00"
```

The standard formatters are mostly intended for machine-readable timestamps. To present dates and times to human readers, use a locale-specific formatter. There are four styles, `SHORT`, `MEDIUM`, `LONG`, and `FULL`, for both date and time—see Table 6.2.

Table 6.2 Locale-Specific Formatting Styles

Style	Date	Time
`SHORT`	`7/16/69`	`9:32 AM`
MEDIUM	Jul 16, 1969	9:32:00 AM
`LONG`	`July 16, 1969`	`9:32:00 AM EDT`
FULL	Wednesday, July 16, 1969	9:32:00 AM EDT

The static methods `ofLocalizedDate`, `ofLocalizedTime`, and `ofLocalizedDateTime` create such a formatter. For example:

Click here to view code image

```
DateTimeFormatter formatter = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.LONG);
String formatted = formatter.format(apollo11launch);
  // July 16, 1969 9:32:00 AM EDT
```

These methods use the default locale. To change to a different locale, simply use the `withLocale` method.

Click here to view code image

```
formatted = formatter.withLocale(Locale.FRENCH).format(apollo11launch);
  // 16 juillet 1969 09:32:00 EDT
```

The `DayOfWeek` and `Month` enumerations have methods `getDisplayName` for giving the names of weekdays and months in different locales and formats.

Click here to view code image

```
for (DayOfWeek w : DayOfWeek.values())
  System.out.print(w.getDisplayName(TextStyle.SHORT, Locale.ENGLISH) + " ");
  // Prints Mon Tue Wed Thu Fri Sat Sun
```

See **Chapter 7** for more information about locales.



Note

The `java.time.format.DateTimeFormatter` class is intended as a replacement for `java.util.DateFormat`. If you need an instance of the latter for backward compatibility, call `formatter.toFormat()`.

Finally, you can roll your own date format by specifying a pattern. For example,

Click here to view code image

```
formatter = DateTimeFormatter.ofPattern("E yyyy-MM-dd HH:mm");
```

formats a date in the form `Wed 1969-07-16 09:32`. Each letter denotes a different time field, and the number of times the letter is repeated selects a particular format, according to rules that are arcane and seem to have organically grown over time. **Table 6.3** shows the most useful pattern elements.

Table 6.3 Commonly Used Formatting Symbols for Date/Time Formats

ChronoField or Purpose	Examples
ERA	`G`: `AD`, `GGGG`: `Anno Domini`, `GGGGG`: `A`
YEAR_OF_ERA	`yy`: `69`, `yyyy`: `1969`
MONTH_OF_YEAR	`M`: `7`, `MM`: `07`, `MMM`: `Jul`, `MMMM`: `July`, `MMMMM`: `J`

ChronoField or Purpose	Examples
DAY_OF_MONTH	d : 6 , dd : 06
DAY_OF_WEEK	e : 3 , E : Wed , EEEE : Wednesday , EEEEE : W
HOUR_OF_DAY	H : 9 , HH : 09
CLOCK_HOUR_OF_AM_PM	K : 9 , KK : 09
AMPM_OF_DAY	a : AM
Period of day (since Java 16)	B : in the evening , at night , midnight
MINUTE_OF_HOUR	mm : 02
SECOND_OF_MINUTE	ss : 00
NANO_OF_SECOND	nnnnnn : 000000
Time zone ID	VV : America/New_York
Time zone name	z : EDT , zzzz : Eastern Daylight Time , v : ET , vvvv : Eastern Time
Zone offset	x : -04 , xx : -0400 , xxx : -04:00 , XXX : same, but use Z for zero
Localized zone offset	O : GMT-4 , OOOO : GMT-04:00

ChronoField or Purpose	Examples
Modified Julian Day	`g` : `58243`

To parse a date/time value from a string, use one of the static `parse` methods. For example,

Click here to view code image

```
LocalDate churchsBirthday = LocalDate.parse("1903-06-14");
ZonedDateTime apollo11launch =
  ZonedDateTime.parse("1969-07-16 03:32:00-0400",
    DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ssxx"));
```

The first call uses the standard `ISO_LOCAL_DATE` formatter, the second one a custom formatter.

The program in **Listing 6.4** shows how to format and parse dates and times.

Listing 6.4 formatting/Formatting.java

Click here to view code image

```
1  package formatting;
2
3  /**
4   * @version 1.0 2016-05-10
```

```
 * @author Cay Horstmann
 */

import java.time.*;
import java.time.format.*;
import java.util.*;

public class Formatting
{
   public static void main(String[] args)
   {
      ZonedDateTime apollo11launch = ZonedDateTime.of(1969, 7, 16, 9, 32, 0, 0,
         ZoneId.of("America/New_York"));

      String formatted = DateTimeFormatter.ISO_OFFSET_DATE_TIME.format(apollo11launch);
      // 1969-07-16T09:32:00-04:00
      System.out.println(formatted);

      DateTimeFormatter formatter = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.LONG);
      formatted = formatter.format(apollo11launch);
      // July 16, 1969 9:32:00 AM EDT
      System.out.println(formatted);
      formatted = formatter.withLocale(Locale.FRENCH).format(apollo11launch);
      // 16 juillet 1969 09:32:00 EDT
      System.out.println(formatted);

      formatter = DateTimeFormatter.ofPattern("E yyyy-MM-dd HH:mm");
      formatted = formatter.format(apollo11launch);
      System.out.println(formatted);

      LocalDate churchsBirthday = LocalDate.parse("1903-06-14");
      System.out.println("churchsBirthday: " + churchsBirthday);
      apollo11launch = ZonedDateTime.parse("1969-07-16 03:32:00-0400",
```

```
38            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ssxx"));
39        System.out.println("apollo11launch: " + apollo11launch);
40
41        for (DayOfWeek w : DayOfWeek.values())
42            System.out.print(w.getDisplayName(TextStyle.SHORT, Locale.ENGLISH) + " ");
43    }
44 }
```

java.time.format.DateTimeFormatter 8

- `String format(TemporalAccessor temporal)`
  formats the given value. The classes `Instant`, `LocalDate`, `LocalTime`, `LocalDateTime`, and `ZonedDateTime`, as well as many others, implement the `TemporalAccessor` interface.
- `static DateTimeFormatter ofLocalizedDate(FormatStyle dateStyle)`
- `static DateTimeFormatter ofLocalizedTime(FormatStyle timeStyle)`
- `static DateTimeFormatter ofLocalizedDateTime(FormatStyle dateTimeStyle)`
- `static DateTimeFormatter ofLocalizedDateTime(FormatStyle dateStyle, FormatStyle timeStyle)`
  yield a formatter for the given styles. The `FormatStyle` enumeration has values `SHORT`, `MEDIUM`, `LONG`, and `FULL`.
- `DateTimeFormatter withLocale(Locale locale)`
  yields a formatter equal to this formatter, using the given locale.
- `static DateTimeFormatter ofPattern(String pattern)`
- `static DateTimeFormatter ofPattern(String pattern, Locale locale)`
  yield a formatter using the given pattern and locale. See **Table 6.3** for the pattern syntax.

java.time.LocalDate 8

- static LocalDate parse(CharSequence text)
- static LocalDate parse(CharSequence text, DateTimeFormatter formatter)
  yield a LocalDate using the default formatter, or the given formatter.

---

java.time.ZonedDateTime 8

- static ZonedDateTime parse(CharSequence text)
- static ZonedDateTime parse(CharSequence text, DateTimeFormatter formatter)
  yield a ZonedDateTime using the default formatter, or the given formatter.

6.7 Interoperating with Legacy Code

As a new creation, the Java Date and Time API will have to interoperate with existing classes—in particular, the ubiquitous java.util.Date , java.util.GregorianCalendar , and java.sql.Date / Time / Timestamp .

The Instant class is a close analog to java.util.Date . In Java 8, that class has two added methods: the toInstant method that converts a Date to an Instant , and the static from method that converts in the other direction.

Similarly, ZonedDateTime is a close analog to java.util.GregorianCalendar , and that class has gained conversion methods in Java 8. The toZonedDateTime method converts a GregorianCalendar to a ZonedDateTime , and the static from method does the opposite conversion.

Another set of conversions is available for the date and time classes in the java.sql package. You can also pass a DateTimeFormatter to legacy code that uses java.text.Format . **Table 6.4** summarizes these conversions.

Table 6.4 Conversions between `java.time` Classes and Legacy Classes

Classes	To Legacy Class	From Legacy Class
`Instant` ↔ `java.util.Date`	`Date.from(instant)`	`date.toInstant()`
`ZonedDateTime` ↔ `java.util.GregorianCalendar`	`GregorianCalendar.from( zonedDateTime)`	`cal.toZonedDateTime()`
`Instant` ↔ `java.sql.Timestamp`	`Timestamp.from(instant)`	`timestamp.toInstant()`
`LocalDateTime` ↔ `java.sql.Timestamp`	`Timestamp.valueOf( localDateTime)`	`timestamp.toLocalDateTime()`
`LocalDate` ↔ `java.sql.Date`	`Date.valueOf(localDate)`	`date.toLocalDate()`
`LocalTime` ↔ `java.sql.Time`	`Time.valueOf(localTime)`	`time.toLocalTime()`
`DateTimeFormatter` → `java.text.DateFormat`	`formatter.toFormat()`	None
`java.util.TimeZone` ↔ `ZoneId`	`TimeZone.getTimeZone(id)`	`timeZone.toZoneId()`
`java.nio.file.attribute.FileTime` ↔ `Instant`	`FileTime.from(instant)`	`fileTime.toInstant()`

You now know how to use the Java 8 date and time library to work with date and time values around the world. The next chapter takes the discussion of programming for an international audience further. You will see how to format program messages, numbers, and currencies in the way that makes sense for your customers, wherever they may be.

Previous chapter

‹ Chapter 5. Database Programming

Next chapter

Chapter 7. Internationalization ›